Sports Player Performance Prediction System

DESIGN, IMPLEMENTATION, AND ANALYSIS

A LOCAL EXPLAINABLE AI APPROACH TO SPORTS ANALYTICS

Jose Temblador, Founder & CEO

CSC-492

Fall 2025

PROJECT OVERVIEW & MOTIVATION

What is it?

A **locally-hosted system** that automatically collects NFL player data to **predict future performance using machine learning.**

Significance

This work has applications in **sports analytics,** serves as a case study for **data engineering** and **machine learning**, and contributes a valuable **open-source tool** to the research community.

The Problem

Being able to **reliably predict** whether a players' performance will be above or below their average.

My Solution

Creating a place where someone can go to see all the **raw stats** along/**relevant statistical data** to **predict player performance**. Bringing together the raw **numbers, trends, tables** into an intuitive interface was the main goal.

TARGET AUDIENCE

The Sports Analyst

Needs accurate, transparent predictions with **explainable features.**

The Data Science Student

Wants **access to raw data and a clear architecture** to experiment with different models.

The Fantasy Sports Player

Requires fast, **easy-to-understand predictions** to make quick lineup decisions

SYSTEM ARCHITECTURE OVERVIEW

Data Source
(nflverse API)

Online API that **stores NFL stats** via the **nflreadpy** library. The basis of the datasets that the models will be trained from.

Data Pipeline
nfl_pipeline.py

An **automated process** that intelligently performs incremental updates. It **checks for the latest local data** and only downloads new, missing weeks, ensuring efficiency.

Local Storage
data/nfl/raw/

All data is stored **locally** as **Parquet** files, a highly efficient columnar format. Raw data is saved using a **standardized naming convention**.

Backend Logic (DataLoader & Feature Engineer)

Handles all **in-memory data operations.** Helper functions **load** and **cache** entire seasons of data, allowing for instantaneous **filtering, sorting, and aggregation** based on user input.

Frontend (Streamlit Dashboard)
app.py

The **Streamlit App** loads these Parquet files into memory using caching for performance. This results in a highly responsive user interface.

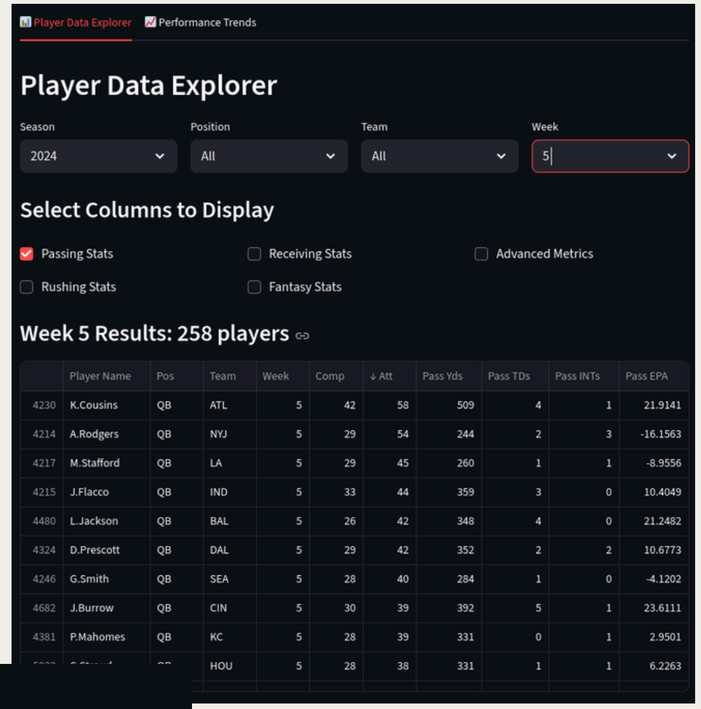
PERFORMANCE & CACHING STRATEGY

- The system is designed to be **highly responsive**.
  - **Efficient Storage (Parquet)**: The columnar format is ideal for reading only the specific data needed for a query, which is much faster than row-based formats like CSV.
- **In-Memory** Caching (Streamlit):
  - Application uses Streamlit's caching to avoid re-reading data/files repeatedly.
  - **@st.cache_resource** is used to initialize the **NFLDataPipeline** only once.
  - **@st.cache_data** is used for the **load_season_data** function. So once a season's data is loaded, it's kept in memory for instant access across all user interactions. The cache is **only** cleared when new data is fetched.

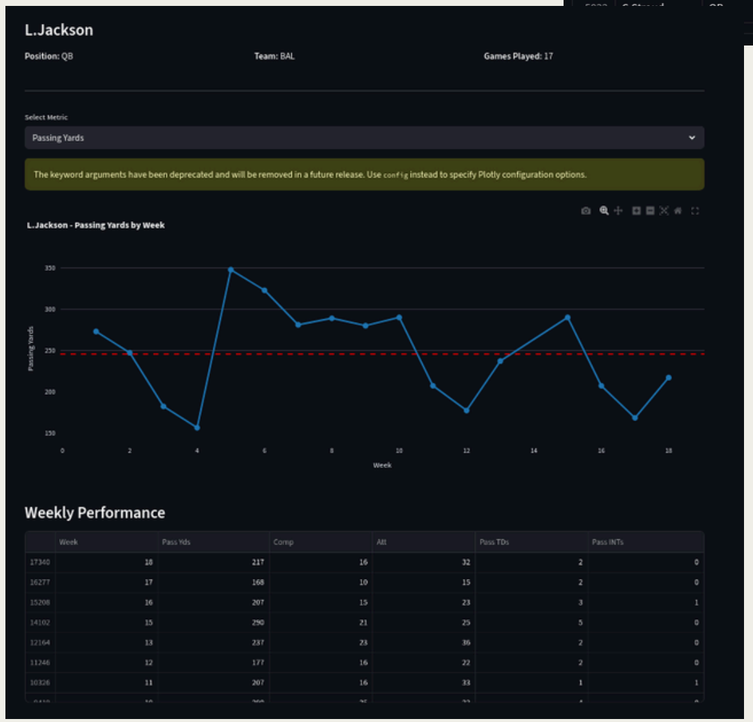
```python
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from pathlib import Path
import sys

# Add src directory to path for imports
sys.path.append(str(Path(__file__).parent / "src"))

from nfl.nfl_pipeline import NFLDataPipeline
from nfl.column_mappings import COLUMN_DISPLAY_NAMES


# Page configuration
st.set_page_config(
    page_title="NFL Analytics Platform",
    page_icon="🏈",
    layout="wide"
)

# Initialize pipeline
@st.cache_resource
def get_pipeline():
    return NFLDataPipeline()

pipeline = get_pipeline()


# Helper function to load all data for a season
@st.cache_data
def load_season_data(season):
    """Load all weeks for a given season"""
    all_data = []

    for week in range(1, 19):  # Weeks 1-18
```

LIVE DEMO

1. Fetching Latest Data
2. Explore Weekly Data
   a. **Player Data Explorer** defaults to most recent data.
   b. **Passing/Rushing/Receiving** stats
3. **Performance Trends** and how players stats are displayed week by week.
   a. Helps to analyze a players **consistency**.



	Player Name	Pos	Team	Week	Comp	↓ Att	Pass Yds	Pass TDs	Pass INTs	Pass EPA
4230	K.Cousins	QB	ATL	5	42	58	509	4	1	21.9141
4214	A.Rodgers	QB	NYJ	5	29	54	244	2	3	-16.1563
4217	M.Stafford	QB	LA	5	29	45	260	1	1	-8.9556
4215	J.Flacco	QB	IND	5	33	44	359	3	0	10.4049
4480	L.Jackson	QB	BAL	5	26	42	348	4	0	21.2482
4324	D.Prescott	QB	DAL	5	29	42	352	2	2	10.6773
4246	G.Smith	QB	SEA	5	28	40	284	1	0	-4.1202
4682	J.Burrow	QB	CIN	5	30	39	392	5	1	23.6111
4381	P.Mahomes	QB	KC	5	28	39	331	0	1	2.9501
			HOU	5	28	38	331	1	1	6.2263



	Week	Pass Yds	Comp	Att	Pass TDs	Pass INTs
17340	18	217	16	32	2	0
16277	17	168	10	15	2	0
15208	16	207	15	23	3	1
14102	15	290	21	25	5	0
12164	13	237	23	36	2	0
11246	12	177	16	22	2	0
10326	11	207	16	33	1	1

PROJECT STATUS & ACCOMPLISHMENTS

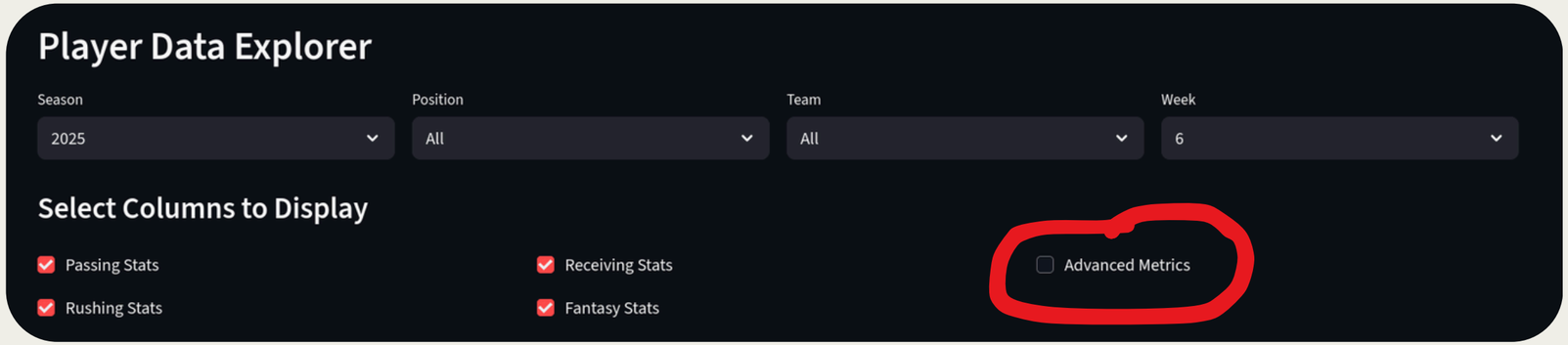
Phase 1 & 2 Complete: The core data infrastructure and interactive dashboard are fully functional and tested.

- Key Accomplishments:
  - **Fully Automated ETL Pipeline – (nfl_pipeline.py)**
  - **High-Performance UI**: Caching in app.py delivers a smooth user experience.
  - **Rich, Local Dataset**: A complete dataset from 2020-2025 is stored locally.

```
.
├── app.py
├── data
│   └── nfl
│       └── cleaned
├── requirements.txt
├── src
│   └── nfl
│       ├── column_mappings.py
│       ├── __init__.py
│       └── nfl_pipeline.py
└── tree.txt
```

FUTURE WORK: MACHINE LEARNING & EXPLAINABLE AI

- Phase 3: Machine Learning Models
  - **Feature Engineering**: A **FeatureEngineer** class to create predictive features **(e.g., rolling averages, opponent strength adjustments)** from the raw stats.
  - **Models (POB & EVOB)**: Implement ML models (XGBoost) to predict not just if a player will do well, **but by how much.**
- Phase 4: Explainable AI & Final Polish
  - **SHAP Analysis**: Integrate **SHAP** to show why a prediction was made.
  - **Natural Language Explanations**: Integration of AI models such as ChatGPT to translate the model's reasoning into plain English.



Thank you!

Jose Temblador, Founder & CEO
CSC-492
Fall 2025